

Reading from Files and Writing to Files in Python

CMSC 104 Section 2
April 8 2026

Administrative Notes

Quiz 3 is next Wednesday, April 15 in class

Same as with previous quizzes:

- Open book, open notes, you can use your computers
- No help from an outside source
- No AI

It covers all material up until today

Reading from and Writing to files in Python

Up until this week all input came from the keyboard and all output went to the screen

- On Monday we introduced the concept of reading from a file
- We'll briefly review that
- And then dive into the topic of writing to files

We'll only use files of text in this class

Python can handle binary files, like pictures

But we're not going to address those cases.

Everything will be ASCII or UTF8 characters. So everything gets read in as strings.

Reminder of the biggest issues reading from files

- Knowing where the file is - its name and path
 - If you try to read from a file that doesn't already exist, your program crashes
 - Spoiler alert: this is not the case when writing to files
- Knowing the formats of data in the file
 - Are there any header rows?
 - Are values separated by commas? Tabs? Spaces?
 - How many values per row?
 - You need to do EDA - "Exploratory Data Analysis" - to assess this

Three methods for reading data

`.read()` - reads all of the remaining data in the file into a single string

`.readlines()` - reads all of the remaining data in the file into a list of strings. Each line of data in the file goes into a separate element in the list

`.readline()` - reads one line of the file into a string

The example we showed on Monday

Using the file “Fall_2025.csv” that’s available on GitHub

When do you use each method?

It's really up to the programmer - what's your comfort level with each method?

- `.readline()` is really useful if you want to get rid of header lines that don't have useful data
- `.read()` gives you a big blob of data that you then have to separate by line
- In this class we'll mostly use `.readlines()`, so that we don't have to break data into a list of lines ourselves

Writing to files

Writing to files

You tell Python that you're writing to a file by opening it with a mode of "w" or "a"

```
with open (filename, "w") as outfile:
```

Modes:

"w" - writing. You will start with an empty file. If the filename you use already exists, it will be erased - all data previously there is gone - and you get an empty file. If the filename does not exist, you get a new, empty file.

"a" - appending. If the filename exists, it is opened, the file pointer is placed at the end of the file, and what you write goes onto the end of the file. Previous data is not deleted. If the filename does NOT already exist, you get a new, empty file

More details:

- The file has to be opened before you can read from it, or write to it
- It has to be closed at the end of use, but our chosen method - “with open” - automatically closes the file
- We’ve studied modes “r”, “w” and “a”. If you leave off the mode, Python defaults to “r.”
 - There are other modes, but we’re not going to get into them

Methods for writing to files

`.write()` - write one string to the file

`.writelines()` - write the elements of a list into a file.

Note: only strings can be written to a file. If you want to save int, float, or boolean variables to a file, you have to convert them to strings first.

This is different from printing to the screen, where you can directly print int's or float's or booleans

There is a “cheat code”

You can “print” to a file using the print statement itself

```
print(x, y, file=outfile)
```

But we won't be using that in this class, because it's actually fairly complicated

“Print” in this case does all the conversion into strings for you - but it might not be in the format you or another user may want

Examples of writing to a file

Can you read from and write to the same file at the same time?

Yes, but only if you're careful.

Remember that when you open the file for writing, any previous data will be erased.

So what you should do is:

- Open the file for reading
- Read in the data
- Manipulate the data as needed
- Prepare the output
- Close the file
- Open the file for writing
- Write your new output

I/O to the same file

If what you want to do is read one line from a file, manipulate that data, write the modification back to the file, and then read the next line

- That's not an easy thing to do with Python